
CS246: Database Management Systems Lab

Week # 03 (1 Questions, 100 Marks)

Lab session: AL1

Held on: 29-Jan-2026 (Monday Timetable)

Lab Timings: 14:00 to 17:00 Hours Pages: 4

Submission dead line: 17:00 hrs, 29-Jan-2026

Submissions accepted from: 16:45 hrs, 29-Jan-2026

Submissions accepted till: 17:30 hrs, 29-Jan-2026

Every one minute delay in submission will result in one mark deduction

Instructors Dr. V. Vijaya Saradhi

Head TA Divanshu Walia

Department of CSE, IIT Guwahati

Question 1: (100 points)

This problem involves analyzing flight delay data from an airport network. The goal is to

1. Process a dataset of flights
2. Compute delays
3. Sort the flights based on delay duration using a custom implementation of quicksort,
4. Generate summary statistics such as average delays per origin airport
5. The top delayed flights.
6. You must implemented the tasks described below using C++.

Input Input file `flights.csv` consists of several records where each record consists of the following.

1. `flight_id` Integer representing a unique identifier, e.g., 1 to 5000.
2. `origin`: String representing a 3-letter airport code, e.g., "DEL" for Delhi.
3. `dest`: String representing a 3-letter airport code, different from origin.
4. `scheduled_time`: String representing date & time whose format is "YYYY-MM-DD HH:MM", e.g., "2023-05-15 14:30".
5. `actual_time`: String representing date & time whose format is "YYYY-MM-DD HH:MM", e.g., "2023-05-15 14:30". This is always after or equal to `scheduled_time`

Total number of records in the file are 5000.

Task 01 (6 marks) Declare a `struct flight_info` to store the input data whose members are

- `int flight_id;`
- `string origin;`
- `string dest;`

- date part of the `scheduled_arrival`;
- time part of the `actual_arrival`;
- int `delay_minutes`;

Task 02 (2 marks) Date and time should be represented in a structure `date_time` whose members are:

- `date`
- `time`

Task 03 (3 marks) The structure `date` consists of the following members (integers):

- Day
- Month
- Year

Task 04 (2 marks) The structure `time` consists of the following members (integers)

- Hours
- Minutes

Task 05 (10 marks) Reading inputs

1. Open the file
2. Skip the header row
3. For each subsequent line:
 - Read the line.
 - Parse the line and extract appropriate fields
 - Parse the date time field corresponding to `scheduled_arrival` and store date in `struct date`; store time in `struct time`
 - Parse the date time field corresponding to `actual_arrival` and store date in `struct date`; store time in `struct time`

Task 06 (10 marks) Compute delay for flight which is the difference between `actual_arrival` and `scheduled_arrival`. To achieve this, write a function `compute_delay` whose specification is:

- **Input argument:** `struct flight_info` containing specific `flight_id`
- **Return type:** `struct time`
- **Functionality:** Compute the flight delay.
- Print the output in a file `delay-computation.csv`.

Task 07 (20 marks) Sort the flights by `delay_minutes` in descending order. To achieve this, implement your own `quicksort` function. Print the result into a file `sorted-delay-info.csv`. To achieve this, implement the function `partition` and `quicksort` whose specifications are given below:

1. (10 marks) `partition` find the array partition
 - (3 marks) **Input arguments**
 - Pointer to the `struct flight_info`

- `int low` (start index)
 - `int high` (end index)
 - (1 mark) **Return type:** `int` (pivot index after partitioning).
 - (6 marks) **Functionality:** Chooses the last element as pivot, partitions the array so elements with greater `delay_minutes` are on the left (for descending sort), and returns the pivot's final position. Swap elements as needed.
2. (15 marks) **quicksort** Sort the array using quick sort algorithm
- (3 marks) **Input arguments**
 - Pointer to the `struct flight_info`
 - `int low` (start index)
 - `int high` (end index)
 - (1 mark) **Return type:** `void`
 - (11 marks) **Functionality:** Recursively sorts the `struct flight_info` array by `delay_minutes` in descending order.
 - Print the sorted output in the file `sorted-delay-info.csv`

Task 08 (25 marks) Compute the aggregate: for each origin airport, the average delay (sum of delays / count of flights from that origin). Print the result into a file `flight-delays-by-origin.csv` whose sample output is shown below.

Implement the function `compute_averages` whose specifications are given below

- (1 mark) **Input argument:** pointer to `struct flight_info`
- (1 mark) **Return type:** `None`
- (23 marks) **Functionality:** Iterate through flights, accumulate sum of delays and count per origin, then compute average (sum / count) for each. Handle division by zero if no flights (though unlikely).
- Print the result into the file `flight-delays-by-origin.csv`

Task 09 (25 marks) Identify the route (origin, destination) pair with the highest average delay. Implement the function `find_most_delayed_route` with the following specification:

- (3 marks) **Input arguments:**
 - Pointer to `struct flight_info` array
 - `int n` (number of flights, $n = 5000$)
- (2 marks) **Return type:** `void`
- (20 marks) **Functionality:**
 - Group the flights by route (origin, dest)
 - For each unique route, compute the **average delay** = (sum of `delay_minutes` for all flights on that route) / (number of flights on that route).
 - Find the route with the **highest average delay**.

- In case of ties (multiple routes with the same highest average), choose the lexicographically smallest route string (e.g., "(AMD, BLR)" before "(BLR, DEL)").
- Write **exactly one line** to the output file `most-delayed-route.csv` in the format:


```
origin,dest,avg_delay_minutes
DEL,BOM,47.8
```
- Use two decimal places for the average delay.

Task 10 (12 marks) Correct output corresponding to the following

- (3 marks) Print the delay computation in the file `delay-computation.csv`.
- (3 marks) Print the sorted delays information in the file `sorted-delay-info.csv`
- (3 marks) Print the flight delays by origin into the file `flight-delays-by-origin.csv`
- (3 marks) Print route with the highest average delay into the file `most-delayed-route.csv`